

GroupNorm#

<https://pytorch.org/docs/stable/generated/torch.nn.GroupNorm.html>

Description:

Applies Group Normalization over a mini-batch of inputs. This layer implements the operation as described in the paper Group Normalization. The input channels are separated into `num_groups` groups, each containing `num_channels / num_groups` channels. `num_channels` must be divisible by `num_groups`. The mean and standard-deviation are calculated separately over each group. γ and β are learnable per-channel affine transform parameter vectors of size `num_channels` if `affine` is `True`. The variance is calculated via the biased estimator, equivalent to `torch.var(input, unbiased=False)`. This layer uses statistics computed from input data in both training and evaluation modes. `num_groups` (int) – number of groups to separate the channels into

Function Signature

```
class torch.nn.GroupNorm(num_groups, num_channels, eps=1e-05, affine=True, device=None, dtype=None)[source]#
```

Parameters

Shape:

Code Examples

```
>>> input = torch.randn(20, 6, 10, 10)
>>> # Separate 6 channels into 3 groups
>>> m = nn.GroupNorm(3, 6)
>>> # Separate 6 channels into 6 groups (equivalent with
InstanceNorm)
>>> m = nn.GroupNorm(6, 6)
>>> # Put all 6 channels into a single group (equivalent with
LayerNorm)
>>> m = nn.GroupNorm(1, 6)
>>> # Activating the module
>>> output = m(input)
```

Detailed Documentation

Documentation

Applies Group Normalization over a mini-batch of inputs.

This layer implements the operation as described in the paper Group Normalization

The input channels are separated into `num_groups` groups, each containing `num_channels / num_groups` channels. `num_channels` must be divisible by `num_groups`. The mean and standard-deviation are calculated separately over each group. γ and β are learnable per-channel affine transform parameter vectors of size `num_channels` if `affine` is `True`. The variance is calculated via the biased estimator, equivalent to `torch.var(input, unbiased=False)`.

This layer uses statistics computed from input data in both training and evaluation modes.

`num_groups` (int) – number of groups to separate the channels into

`num_channels` (int) – number of channels expected in input

`eps` (float) – a value added to the denominator for numerical stability. Default: `1e-5`

`affine` (bool) – a boolean value that when set to `True`, this module has learnable per-channel affine parameters initialized to ones (for weights) and zeros (for biases). Default: `True`.

Input: $(N, C, *)$ $(N, C, *)$ where $C = \text{num_channels}$

Output: $(N, C, *)$ (same shape as input)